

# 50 Exercícios de C++ — Do Básico ao Avançado

Lista pensada para quem está começando em C++ e quer treinar **lógica de programação** ao mesmo tempo em que aprende a linguagem. Faça na ordem: cada bloco usa conceitos do anterior.

**Dica de estudo:** tente resolver sozinho antes de procurar a resposta. Se travar por mais de 20 minutos, pesquise só o conceito específico (não a solução pronta) e volte a tentar.

---

## NÍVEL BÁSICO

### Entrada, Saída e Variáveis

Aqui você aprende `cout`, `cin`, tipos de dados (`int`, `float`, `double`, `char`, `string`) e operações matemáticas simples.

1. Escreva um programa que exiba na tela a mensagem `Olá, Mundo!`.
2. Leia o nome e a idade do usuário e exiba: `Olá, [nome]! Você tem [idade] anos.`
3. Leia dois números inteiros e mostre a soma, a subtração, a multiplicação e a divisão entre eles.
4. Leia a base e a altura de um retângulo e calcule sua área.
5. Leia uma temperatura em graus Celsius e converta para Fahrenheit. (Fórmula:  $F = C * 9/5 + 32$ )
6. Leia três notas de um aluno e calcule a média aritmética.
7. Leia dois valores (A e B) e troque o conteúdo das variáveis, exibindo antes e depois.
8. Leia o raio de um círculo e calcule sua área e seu perímetro. (Use `3.14159` para  $\pi$ )
9. Leia o preço de um produto e aplique um desconto de 15%, mostrando o valor final.
10. Leia um valor em segundos e converta para horas, minutos e segundos.

### Condicionais (if / else / switch)

Agora entram as decisões: operadores relacionais (`>`, `<`, `==`, `!=`) e lógicos (`&&`, `||`, `!`).

11. Leia um número e diga se ele é **par** ou **ímpar**.
12. Leia um número e informe se ele é **positivo**, **negativo** ou **igual a zero**.
13. Leia dois números e mostre qual é o maior (ou avise se são iguais).
14. Leia três números e mostre o maior deles.

15. Leia um ano e verifique se ele é **bissexto**. (Bissexto: divisível por 4, exceto os múltiplos de 100 que não sejam de 400)
16. Leia o peso e a altura de uma pessoa, calcule o **IMC** e classifique (abaixo do peso, normal, sobrepeso, obesidade).
17. Leia a média de um aluno e informe se ele foi **Aprovado** ( $\geq 7$ ), está em **Recuperação** ( $\geq 5$  e  $< 7$ ) ou **Reprovado** ( $< 5$ ).
18. Leia a idade de uma pessoa e classifique: criança (0–12), adolescente (13–17), adulto (18–59) ou idoso (60+).
19. Faça uma calculadora que leia dois números e uma operação (+, -, \*, /) e mostre o resultado. Use **switch**.
20. Leia três lados de um triângulo e diga se ele é **equilátero**, **isósceles** ou **escaleno** (ou se não forma um triângulo).

## Estruturas de Repetição (for / while / do-while)

O coração da lógica de programação. Repetir tarefas, acumular resultados, contar.

21. Exiba todos os números de 1 a 100.
22. Exiba todos os números **pares** entre 1 e 50.
23. Leia um número e exiba sua **tabuada** (de 1 a 10).
24. Leia um número N e calcule a soma de todos os números de 1 até N.
25. Leia um número e calcule seu **fatorial** (ex:  $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ ).
26. Exiba os 15 primeiros números da sequência de **Fibonacci** (0, 1, 1, 2, 3, 5, 8...).
27. Leia um número e verifique se ele é **primo**.
28. Leia um número e calcule a **soma dos seus dígitos** (ex:  $1234 \rightarrow 1+2+3+4 = 10$ ).
29. Leia um número inteiro e exiba-o **invertido** (ex:  $123 \rightarrow 321$ ).
30. Leia vários números até que o usuário digite 0. No final, exiba quantos foram digitados e a média deles.

---

## NÍVEL INTERMEDIÁRIO

### Funções

Aprenda a organizar seu código em blocos reutilizáveis, com parâmetros e retorno. Alguns exercícios pedem **recursão**.

31. Crie uma função **maior(int a, int b)** que retorne o maior entre dois números.
32. Crie uma função que calcule o fatorial de um número usando **recursão**.
33. Crie uma função **bool ehPrimo(int n)** que retorne se um número é primo.
34. Crie uma função **potencia(base, expoente)** que calcule a potência sem usar bibliotecas prontas.

35. Crie uma função **recursiva** que retorne o N-ésimo termo de Fibonacci.
36. Crie uma função que receba uma **string** e retorne quantas **vogais** ela contém.
37. Crie uma função que receba um número e retorne **true** se ele for **perfeito** (a soma dos seus divisores, exceto ele mesmo, é igual a ele — ex:  $6 = 1+2+3$ ).
38. Crie uma função que calcule o **MDC** (Máximo Divisor Comum) entre dois números.

## Vetores (Arrays) e Strings

Trabalhando com coleções de dados. Percorrer, buscar, ordenar.

39. Leia 10 números em um vetor e mostre o **maior** e o **menor** deles.
40. Leia 10 números em um vetor e calcule a **média** dos valores.
41. Leia um vetor de 10 posições e exiba-o na **ordem inversa**.
42. Faça uma **busca linear**: leia um vetor e um valor, e diga se (e em qual posição) esse valor aparece.
43. Leia um vetor e conte quantas vezes um determinado valor aparece nele.
44. Ordene um vetor de números em ordem crescente usando o algoritmo **Bubble Sort**.
45. Leia uma palavra e verifique se ela é um **palíndromo** (lê-se igual de trás pra frente — ex: "arara").
46. Leia uma frase e conte quantas **palavras** ela possui.

---

## NÍVEL AVANÇADO

### Matrizes e Structs

Dados bidimensionais e a criação dos seus próprios tipos de dados.

47. Leia duas matrizes 3×3 e calcule a **soma** delas, exibindo a matriz resultante.
48. Leia uma matriz 3×3 e exiba a sua **transposta** (linhas viram colunas).

### Programação Orientada a Objetos (POO)

Introdução a classes, objetos, atributos e métodos — o próximo grande passo em C++.

49. Crie uma **struct** (ou **class**) **Aluno** com nome e três notas. Leia os dados de vários alunos e exiba o nome e a média de cada um.
  50. Crie uma classe **ContaBancaria** com atributos **saldo** e **titular**, e métodos **depositar()**, **sacar()** (que não permita saldo negativo) e **exibirSaldo()**. Faça um pequeno menu para testar todas as operações.
-

## Como aproveitar melhor esta lista

- **Compile e teste sempre.** Não confie só na leitura do código — rode e veja o resultado.
- **Comece simples e vá refinando.** Faça funcionar primeiro, depois melhore (validação de entrada, mensagens mais claras, etc.).
- **Comente seu raciocínio.** Escrever comentários ajuda a fixar a lógica.
- **Revisite os antigos.** Depois de aprender funções, tente reescrever exercícios do início usando funções. Depois de aprender POO, reescreva usando classes.
- **Erre à vontade.** Cada erro de compilação é uma lição sobre como a linguagem funciona.

Bons estudos! 